Data Structures (ETCCDS202)

Unit–1 Assignment: Foundations & Algorithmic Analysis

Course Details

Name of the School: School of Engineering & Technology
Program: B.Tech (CSE-AI/ML)
Semester: 1 Batch: 2025-29
Course Title: Data Structures
Course Code: ETCCDS202
Student Name: Payal
Roll No.: 2501730224
Section: A
Submission Date: 5/03/2026

1. What is an Abstract Data Type (ADT)?
An **Abstract Data Type (ADT)** is a logical or mathematical model of a data structure that defines the type of data stored and the operations that can be performed on it, without specifying the implementation details.
In simple terms, ADT tells **what operations can be performed**, but it does not tell **how those operations are implemented internally**.
For example, a **Stack ADT** defines the following operations:

- push(x) → insert an element into the stack
- pop() → remove the top element
- peek() → view the top element
- is_empty() → check whether the stack is empty
- size() → return the number of elements in the stack

The internal implementation may use an **array or linked list**, but the operations remain the same. Thus, ADT separates the **interface** from the **implementation**.

---

2. Why are stack operations usually O(1)?
Most stack operations such as **push, pop, and peek** take constant time.
Reason
In a stack, elements are inserted and removed only from the **top of the stack**. Since there is no need to traverse the entire structure, the operation takes the same amount of time regardless of the number of elements.
For example:
Push operation simply adds an element at the end of the list.
Pop operation removes the last inserted element.
Peek only reads the last element.
Because these operations involve only a single step, the time complexity is **O(1)**.

---

3. Explain recursive factorial and its complexity.
The factorial of a number **n** is defined as the product of all positive integers less than or equal to n.
Example:
$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$
The recursive definition of factorial is:
$n! = n \times (n - 1)!$
$0! = 1$
Here, **0! = 1** acts as the base case that stops recursion.
Time Complexity
The factorial function makes **n recursive calls** before reaching the base case.
Therefore:

Time Complexity = **O(n)**
Best Case = O(1) when n = 0
Worst Case = O(n)
Big-Ω = O(n)
Big-Θ = O(n)
Space Complexity
Since recursion uses the call stack to store function calls, the stack depth becomes n.
Space Complexity = **O(n)**

---

4. Explain naive Fibonacci and its complexity.
The Fibonacci sequence is defined as:
F(0) = 0
F(1) = 1
F(n) = F(n − 1) + F(n − 2)
Example sequence:
0, 1, 1, 2, 3, 5, 8, 13, 21 …
In the **naive recursive approach**, each function call generates two more recursive calls.
For example:
F(5) = F(4) + F(3)
But F(4) again calls F(3) and F(2), which means the same values are recomputed many times.
Time Complexity
Because each call produces two more calls, the recursion tree grows exponentially.
Time Complexity = **$O(2^n)$**
Space Complexity
The maximum depth of recursion is n.
Space Complexity = **O(n)**
Why naive Fibonacci becomes slow for large n
The main reason is **repeated calculations**. The algorithm recomputes the same Fibonacci values many times, which significantly increases the number of recursive calls.

---

5. Explain memoized Fibonacci and its complexity.
Memoization is an optimization technique used to store previously computed results so that they can be reused when needed again.

In memoized Fibonacci, once a Fibonacci value is calculated, it is stored in a memory structure such as a dictionary. If the same value is required again, the program retrieves it directly instead of recalculating it.
This removes redundant recursive calls.

Time Complexity

Each Fibonacci value is computed only once.
Therefore:
Time Complexity = **O(n)**

Space Complexity

The program stores Fibonacci values in memory and also uses recursion stack.
Space Complexity = **O(n)**
Memoization greatly improves performance compared to naive recursion.

---

6. Explain Tower of Hanoi and its complexity.

Tower of Hanoi is a classic recursive problem involving three rods and n disks of different sizes.
Rules of the problem:

1. Only one disk can be moved at a time.
2. A larger disk cannot be placed on a smaller disk.
3. All disks must be moved from source rod to destination rod.

Recursive Approach

To move n disks:

1. Move $n - 1$ disks from source to auxiliary rod.
2. Move the largest disk to the destination rod.
3. Move the $n - 1$ disks from auxiliary rod to destination rod.

Number of Moves

The minimum number of moves required is:
$2^n - 1$
For example:
If $n = 3$
Moves = 7

Time Complexity

The number of operations doubles for each additional disk.
Time Complexity = **$O(2^n)$**

Space Complexity

Recursion depth is equal to n.
Space Complexity = **O(n)**

---

7. Explain recursive binary search and its complexity.

Binary search is a searching technique used to find an element in a **sorted array**.
Instead of checking each element one by one, binary search repeatedly divides the array into two halves.

Working

1. Find the middle element of the array.
2. Compare it with the target value.
3. If equal, return the index.
4. If the target is smaller, search the left half.
5. If the target is larger, search the right half.

This process continues until the element is found or the search space becomes empty.

Recurrence Relation

$T(n) = T(n/2) + O(1)$

This means each recursive call reduces the problem size by half.

Time Complexity

Best Case = **O(1)** (element found in the middle)
Average Case = **O(log n)**
Worst Case = **O(log n)**

Space Complexity

Because recursion is used, the call stack depth becomes log n.
Space Complexity = **O(log n)**

---

8. Why is Binary Search a Divide & Conquer algorithm?

Binary search follows the **Divide & Conquer paradigm** because it divides the problem into smaller subproblems.
Steps:

1. Divide the array into two halves.
2. Decide which half may contain the element.
3. Recursively search only that half.

Since the search space is halved at every step, it efficiently reduces the number of comparisons.

---

9. Why is memoized Fibonacci a Dynamic Programming approach?

Dynamic Programming is a technique used to solve problems that contain **overlapping subproblems**.
In naive Fibonacci, the same Fibonacci values are computed multiple times.
Memoization solves this issue by storing previously computed values and reusing them when needed.
Thus, memoized Fibonacci follows the **Dynamic Programming paradigm**.

10. Real-life examples of algorithm paradigms

Divide & Conquer Example

Searching a word in a dictionary. Instead of reading from the first page, we open the middle page and decide which half to search next.

Dynamic Programming Example

Saving results of previously solved calculations in a notebook so that we do not need to solve them again.

Greedy Example

While giving change in currency, we usually give the **largest denomination first**, such as 500, then 200, then 100. This reduces the number of notes required.